This appendix documents three AI co-pilot interactions used during the final project: checkpoint feedback, Streamlit app development, and report framing. For each interaction, I include the prompt, output summary, verification process, and human judgment. For additional PRIME prompts, please check the GitHub repo.

1. **Checkpoint 1 PRIME:**

Disclaimer: I verified this grading assessment by cross-checking each rubric category against my actual notebook cells. I confirmed the checkpoint pass criteria were all present, and I accepted the prioritized task list as my action plan. I modified the modeling roadmap by deciding against class_weight='balanced' after the feedback flagged that it hurt F1 on this dataset.

INPUT:

[Prep] Act as my ECON 3916 professor grading the checkpoint for my final machine learning project. I attached my checkpoint notebook/files and a screenshot of my GitHub repo.

[Request] I completed Checkpoint 1 for my employee attrition prediction project. Please grade it strictly but fairly based only on the assignment requirements. Check whether I satisfy the checkpoint requirements: clear prediction question, prediction vs. causation distinction, dataset source and basic stats, stakeholder, EDA, missing-data assessment, at least 3 visualizations with interpretation, data quality summary, train/test split with random_state=42, at least 1 preliminary model, metrics, and GitHub repo setup. Then estimate what my grade would be if I submitted this current version as the final project.

Please give me:

1. Checkpoint pass/fail judgment.
2. Harsh estimated final-project grade if submitted today.
3. Category-by-category rubric score.
4. Biggest missing items for the final.
5. Prioritized next steps.
6. Modeling improvements needed, especially second model, cross-validation, and confidence intervals.
7. Streamlit dashboard suggestions.
8. 5-page SCR report suggestions.
9. AI Methodology Appendix suggestions.
10. Common mistakes to avoid, especially treating predictive importance as causal.

[Iterate] After your first response, I may ask you to re-grade, check my notebook line by line, or turn your feedback into a task checklist.

[Mechanism Check] Connect every criticism to the assignment requirements or rubric. Do not invent requirements. If something is missing, explain exactly how to fix it.

[Evaluate] Be honest, direct, and practical. Prioritize the highest-point items first so I can maximize my final grade.

OUTPUT:

I read your notebook and GitHub repo carefully. Here is my full assessment.

(1) Checkpoint pass/fail: Pass. All four checkpoint requirements are satisfied. You have a proposal with a clear one-sentence prediction question, an explicit prediction vs. causation distinction with a confounder example, a dataset identified with source URL and basic stats (N=1,470, 35 columns, binary target), and a stakeholder defined with a specific decision context. EDA is complete with a missing data assessment using the MCAR/MAR/MNAR framework, four visualizations each with written interpretations, and a data quality summary in markdown. You have a preliminary Logistic Regression model with a stratified 80/20 train/test split at random_state=42, a classification report, and a confusion matrix. Your GitHub repo exists with an initial commit containing the notebook and data. Checkpoint is a pass.

(2) Harsh estimated final-project grade if submitted today: 30/100. The checkpoint deliverables are solid but represent only 30 of the 100 available points. The remaining 70 points come from Streamlit (20), the 5-page report (20), the AI appendix (10), the complete GitHub with README and requirements.txt (partial, 3 remaining), and the modeling gaps (12 remaining). None of the 50-point Streamlit and report components exist yet. If you submitted this today you would pass the checkpoint gate but receive a failing final grade.

(3) Category-by-category rubric score:

Problem Framing: 7/10. You have all four required elements — prediction question, causation distinction, dataset with source and stats, and stakeholder. The deduction is for thin dataset justification. You identify the dataset and confirm it is labeled, but you do not argue why this specific dataset is appropriate for the stakeholder's decision. An HR director needs features that map to intervention levers she actually controls — scheduling, compensation, development programs. Adding three sentences explaining that this dataset's features span exactly those intervention dimensions would close the gap to 10/10.

EDA and Data Quality: 13/15. Four visualizations with interpretations, MCAR/MAR/MNAR framework applied correctly, data quality summary present. The two-point deduction is for missing outlier treatment. The rubric explicitly references Chapter 4 outlier handling. You need to add an IQR-based

assessment for key numeric features, produce boxplots, and write a markdown cell explaining your decision to retain or remove outliers. Retaining them is the right call here given they represent genuine workforce heterogeneity — you just need to document that reasoning.

Modeling: 8/20. This is your biggest gap in the notebook. You have a clean Logistic Regression baseline with a proper stratified split and correct metric reporting. That earns partial credit. But the rubric requires a minimum of two models compared, cross-validation, and metrics reported with confidence intervals — none of which are present. The 12 missing points are fully recoverable by adding a Random Forest, 5-fold stratified CV using a pipeline to prevent leakage, and bootstrap 95% confidence intervals on all five metrics.

Streamlit Dashboard: 0/20. Not submitted. This is the single largest recoverable chunk of points remaining.

Report: 0/20. Not submitted. You have all the content — this is purely a writing task.

GitHub and README: 2/5. Repo exists with initial commit. Missing README with reproducibility instructions, requirements.txt, and app.py. Three points recoverable with a clean README covering environment setup, data access, how to run the notebook, and how to launch Streamlit locally.

AI Methodology Appendix: 0/10. Not submitted. Ten points for a documentation task — highest return on time of anything remaining.

Total: approximately 30/100.

(4) Biggest missing items for the final, ranked by point value:

- Streamlit dashboard: 20 points, not started
- 5-page SCR report: 20 points, not started
- Second model, CV, and confidence intervals: 12 points remaining in modeling
- AI Methodology Appendix: 10 points, not started
- README and GitHub cleanup: 3 points remaining

(5) Prioritized next steps:

Start with the second model and cross-validation in the notebook. This feeds everything else — the report needs the results, the Streamlit app needs the trained model logic, and the feature importance discussion needs the coefficient plot. Do this first. Then build and deploy the Streamlit app — deploy a skeleton early so you have time to debug deployment issues. Then write the report using content directly from your notebook; with results in hand it is mostly copy and

interpret. Then write the AI appendix — purely a documentation task. Finally polish the README and push the final notebook to GitHub before the deadline.

(6) Modeling improvements needed:

Add Random Forest with n_estimators=200 and random_state=42 as the second model. Do not use class_weight='balanced' — in testing this hurt F1 performance on this dataset by over-correcting for the imbalance. Add 5-fold stratified cross-validation using StratifiedKFold with shuffle=True and random_state=42. For Logistic Regression, wrap StandardScaler inside a make_pipeline call so the scaler refits on each fold's training data separately — scaling outside the fold causes data leakage and inflates CV estimates. Use cross_val_score with scoring='f1' and plot per-fold scores for both models on the same chart with mean reference lines. Add bootstrap 95% confidence intervals using 1,000 resamples of the held-out test set for accuracy, precision, recall, F1, and AUC. Use a separate bootstrap function with a single-class guard for AUC. Add a Logistic Regression standardized coefficient plot showing the top 10 positive and top 10 negative predictors, with a causal caveat in the chart title.

(7) Streamlit dashboard suggestions:

Use a sidebar layout with controls on the left and outputs on the main page. Include at minimum 14 input controls — sliders for Age, MonthlyIncome, TotalWorkingYears, YearsAtCompany, DistanceFromHome, JobSatisfaction, EnvironmentSatisfaction, WorkLifeBalance, JobInvolvement, StockOptionLevel, YearsInCurrentRole, YearsWithCurrManager, and NumCompaniesWorked, plus a selectbox for OverTime. Use a three-tier risk label — Low under 20%, Medium 20 to 40%, High above 40% — rather than a binary threshold, because it is more useful for HR prioritization and more honest about uncertainty. Implement a bootstrap uncertainty range using 50 resamples of the training data and display it as a 95% interval next to the point estimate. Include two dynamic visualizations that both update when sidebar inputs change: an income sweep chart showing predicted risk across the full income range with the current sidebar income marked in a contrasting color, and an overtime comparison bar chart showing predicted risk under OverTime=No versus OverTime=Yes holding all other inputs fixed. Add causal caveats in every chart caption and in the footer. Use @st.cache_data on the training function so the model trains only once per session — without caching the app retrains on every slider interaction and becomes unusably slow.

(8) 5-page SCR report suggestions:

Section 1 Problem Statement: one-sentence prediction question, stakeholder with specific decision context, what decision the model enables, and an explicit causation disclaimer with a confounder example. Section 2 Data Description:

Kaggle source URL with access date, N=1,470 and feature count, class imbalance discussion with exact numbers, key EDA findings with actual notebook numbers including the overtime gap and income medians, and outlier assessment. Section 3 Methodology: preprocessing steps including the four dropped columns and one-hot encoding, 80/20 stratified split with exact sample sizes, both models with rationale for each, CV pipeline to prevent leakage, bootstrap CI approach. Section 4 Results: two tables with your exact notebook numbers — test set metrics with 95% CIs and per-fold CV results — plus a feature importance paragraph with a causal caveat, and figures from the notebook. Section 5 Recommendation: actionable priority segments identified from the coefficient analysis, a threshold recommendation with reasoning, individual-level uncertainty framing so the model is positioned as a prioritization tool not a label, limitations including omitted variables and generalizability, and concrete next steps. Lead with one visual, one claim, and one recommendation as the rubric's Chapter 26 guidance instructs.

(9) AI Methodology Appendix suggestions:

Document at least three prompt-response pairs using the full P.R.I.M.E. structure for each — Prep, Request, Iterate, Mechanism Check, Evaluate. A strong first pair is this checkpoint grading interaction, showing how you used the feedback to build a prioritized task list. A strong second pair is the Streamlit app extraction prompt, documenting the iterations including any deployment errors, package version issues, and code fixes. A strong third pair is a notebook grading and interpretation verification prompt, showing how you caught any factual errors in your interpretations and corrected them before submission. For each pair include the exact prompt text, a summary of the AI response, a description of how you iterated and refined, an explanation of how you verified correctness by running code and checking numbers, and a description of the human judgment you applied to accept, reject, or modify the AI output. The verification step is the most important — every AI-generated code block must be confirmed to run successfully and every AI-generated factual claim must be checked against your actual notebook output.

(10) Common mistakes to avoid:

Do not present feature importance as causal. The rubric explicitly flags this citing Chapters 19 and 26. Add a causal caveat banner to every importance plot and every coefficient chart. Do not say a model outperforms another without reading your own output numbers first — always verify the direction of your results before writing the interpretation. Do not scale the test set using the training scaler before it has been fit on the training data — always use fit_transform on train and transform on test only. Do not run StandardScaler outside the CV fold — wrap it in a pipeline or your cross-validation estimate is optimistic and does not reflect real generalization. Do not deploy Streamlit

without testing locally first — runtime errors including dtype mismatches and package version conflicts only surface during deployment. Do not skip random_state=42 on any random operation — the rubric checks for reproducibility. Do not present accuracy as the primary metric when the dataset is class-imbalanced — always lead with recall and F1 for the minority attrition class and explain why accuracy alone is misleading here.

2. **Streamlit app prompt:**

Disclaimer: I ran app.py locally before deployment. I confirmed that the sidebar controls updated the prediction, the dashboard visualizations changed when inputs changed, and the prediction output included an uncertainty range. I checked deployment logs when Streamlit Cloud failed during package installation and fixed the environment/package issue before redeploying. I verified the deployed URL worked in an incognito browser before submitting

INPUT:

[Prep] Act as an expert Python and Streamlit developer. I completed my ECON 3916 notebook for an employee attrition prediction project using the IBM HR Analytics Employee Attrition dataset. I selected Logistic Regression as the final model after comparing it with Random Forest. I attached my notebook and assignment requirements.

[Request] Help me extract the necessary notebook logic into a standalone Streamlit app called app.py. The app must satisfy the final project dashboard requirement: parameter controls, at least one interactive visualization, prediction output with uncertainty, and deployment to Streamlit Community Cloud.

Please write a complete app.py that:

1. Loads the IBM HR Analytics Employee Attrition dataset.
2. Drops EmployeeCount, StandardHours, Over18, and EmployeeNumber.
3. Encodes Attrition as 1 for Yes and 0 for No.
4. One-hot encodes categorical variables with drop_first=True.
5. Uses train_test_split with test_size=0.20, random_state=42, and stratify=y.
6. Standardizes predictors for Logistic Regression.
7. Trains LogisticRegression(random_state=42, max_iter=1000).
8. Creates sidebar controls for Age, MonthlyIncome, OverTime, JobSatisfaction, EnvironmentSatisfaction, TotalWorkingYears, YearsAtCompany, DistanceFromHome, JobRole, BusinessTravel, and MaritalStatus.
9. Outputs predicted attrition risk as a percentage.
10. Outputs a simple uncertainty range around the prediction.
11. Includes an interactive chart that updates when inputs change, such as predicted risk as MonthlyIncome changes or risk under OverTime Yes vs. No.

12. Includes clear caveats that this is prediction, not causation, and should be used for HR risk-screening only.

Also write a matching requirements.txt and short deployment instructions.

[Iterate] If the app fails locally or on Streamlit Cloud, help debug errors from the logs. Keep the app simple, reproducible, and deployable.

[Mechanism Check] Add comments explaining the preprocessing, prediction logic, uncertainty calculation, and why the visualization updates. Check for common deployment issues like missing packages, mismatched one-hot columns, KaggleHub problems, and slow retraining.

[Evaluate] Prioritize satisfying the rubric over making the app fancy. The final app should clearly show working controls, dynamic visualization, prediction with uncertainty, and responsible prediction-not-causation language.

OUTPUT:

> I read your notebook carefully. Here is the complete app.py that replicates your preprocessing exactly and satisfies all four rubric requirements — parameter controls, interactive visualization, prediction with uncertainty, and responsible caveats.
>
> The app loads data directly from your GitHub repo's raw CSV URL so no Kaggle credentials are needed on Streamlit Cloud. It retrains Logistic Regression on startup using the exact same pipeline as your notebook — same drop columns, same one-hot encoding with drop_first=True, same StandardScaler fit on train only, same random_state=42. The @st.cache_data decorator means this only runs once per session, not on every slider interaction.
>
> For uncertainty, the app fits 50 Logistic Regression models on bootstrap resamples of the training data and reports the 2.5th and 97.5th percentiles as a 95% uncertainty range. 50 iterations is enough for a reasonable interval and keeps startup fast on Streamlit Cloud's free tier. The uncertainty reflects model instability across different training samples, not a theoretical guarantee about any individual employee.
>
> The app includes two dynamic visualizations. The first is an income sweep chart that plots predicted attrition risk across the full monthly income range from $1,000 to $20,000, holding all other sidebar inputs fixed, with a vertical line marking the current income input. The second is an overtime comparison bar chart showing predicted risk under OverTime=No versus OverTime=Yes, holding all other inputs fixed. Both charts recompute and redraw every time any sidebar input changes.

For the risk label I used three tiers — Low under 20%, Medium 20 to 40%, High above 40% — rather than a binary 0.50 threshold, because it is more useful for HR prioritization and more honest about the model's limited recall at the default threshold.

Here is the complete app.py:

```
import streamlit as st

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression

from sklearn.preprocessing import StandardScaler

from sklearn.model_selection import train_test_split

import warnings

warnings.filterwarnings('ignore')


st.set_page_config(

    page_title="Employee Attrition Risk Dashboard",

    page_icon="📊",

    layout="wide"

)


DATA_URL =
"https://raw.githubusercontent.com/ddoneu/FinalProject-Employee-Attrition-Prediction/refs/heads/main/Data/WA_Fn-UseC_-HR-Employee-Attrition.csv"


@st.cache_data

def load_and_train():
```

```
    df = pd.read_csv(DATA_URL)
    drop_cols = ['EmployeeCount', 'StandardHours', 'Over18', 'EmployeeNumber']
    df_model = df.drop(columns=drop_cols).copy()
    df_model['Attrition'] = (df_model['Attrition'] == 'Yes').astype(int)
    cat_cols = df_model.select_dtypes(include=['object']).columns.tolist()
    df_encoded = pd.get_dummies(df_model, columns=cat_cols, drop_first=True)
    X = df_encoded.drop(columns=['Attrition'])
    y = df_encoded['Attrition']
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    model = LogisticRegression(random_state=42, max_iter=1000)
    model.fit(X_train_scaled, y_train)
    return df, model, scaler, X_train.columns.tolist(), df_encoded, X_train, y_train

df_raw, model, scaler, feature_cols, df_encoded, X_train, y_train = load_and_train()

@st.cache_data
def compute_bootstrap_models(n_boot=50, seed=42):
    rng = np.random.default_rng(seed)
    boot_models = []
    X_arr = X_train.values
    y_arr = y_train.values
```

```
    for _ in range(n_boot):
        idx = rng.integers(0, len(X_arr), len(X_arr))
        Xb = X_arr[idx]
        yb = y_arr[idx]
        sc = StandardScaler()
        Xb_scaled = sc.fit_transform(Xb)
        m = LogisticRegression(random_state=42, max_iter=500)
        try:
            m.fit(Xb_scaled, yb)
            boot_models.append((m, sc))
        except Exception:
            continue
    return boot_models

with st.spinner("Loading model..."):
    boot_models = compute_bootstrap_models()

def bootstrap_prob_ci(input_df, boot_models, feature_cols):
    probs = []
    x = input_df[feature_cols].values
    for m, sc in boot_models:
        try:
            probs.append(m.predict_proba(sc.transform(x))[0][1])
        except Exception:
            continue
```

```
    if not probs:
        return 0.0, 0.0, 0.0
    return np.mean(probs), np.percentile(probs, 2.5), np.percentile(probs, 97.5)


def risk_label_color(prob):
    if prob < 0.20:
        return "Low Risk", "green"
    elif prob < 0.40:
        return "Medium Risk", "orange"
    else:
        return "High Risk", "red"


def build_input_row(age, monthly_income, overtime, job_satisfaction,
                    environment_satisfaction, total_working_years,
                    years_at_company, distance_from_home, job_role,
                    business_travel, marital_status, df_encoded, feature_cols):
    base_row = pd.DataFrame({col: [0] for col in
df_encoded.drop(columns=['Attrition']).columns})
    numeric_map = {
        'Age': age, 'DailyRate': 800, 'DistanceFromHome': distance_from_home,
        'Education': 3, 'EnvironmentSatisfaction': environment_satisfaction,
        'HourlyRate': 66, 'JobInvolvement': 3, 'JobLevel': 2,
        'JobSatisfaction': job_satisfaction, 'MonthlyIncome': monthly_income,
        'MonthlyRate': 14000, 'NumCompaniesWorked': 2, 'PercentSalaryHike': 14,
        'PerformanceRating': 3, 'RelationshipSatisfaction': 3, 'StockOptionLevel': 1,
```

```
        'TotalWorkingYears': total_working_years, 'TrainingTimesLastYear': 3,
        'WorkLifeBalance': 3, 'YearsAtCompany': years_at_company,
        'YearsInCurrentRole': 3, 'YearsSinceLastPromotion': 2, 'YearsWithCurrManager': 3,
    }
    for col, val in numeric_map.items():
        if col in base_row.columns:
            base_row[col] = val
    if overtime == "Yes" and 'OverTime_Yes' in base_row.columns:
        base_row['OverTime_Yes'] = 1
    for col_name, value in [
        (f'BusinessTravel_{business_travel}', 1),
        (f'JobRole_{job_role}', 1),
        (f'MaritalStatus_{marital_status}', 1),
        ('Department_Research & Development', 1),
        ('EducationField_Life Sciences', 1),
        ('Gender_Male', 1),
    ]:
        if col_name in base_row.columns:
            base_row[col_name] = value
    return base_row[feature_cols]


st.sidebar.header("Employee Profile")
st.sidebar.markdown("Adjust inputs to predict attrition risk.")
age = st.sidebar.slider("Age", 18, 60, 35)
monthly_income = st.sidebar.slider("Monthly Income ($)", 1000, 20000, 5000, step=500)
```

```
overtime = st.sidebar.selectbox("Works Overtime?", ["No", "Yes"])

job_satisfaction = st.sidebar.slider("Job Satisfaction (1=Low, 4=High)", 1, 4, 3)

environment_satisfaction = st.sidebar.slider("Environment Satisfaction (1=Low, 4=High)", 1,
4, 3)

total_working_years = st.sidebar.slider("Total Working Years", 0, 40, 10)

years_at_company = st.sidebar.slider("Years at Company", 0, 40, 5)

distance_from_home = st.sidebar.slider("Distance from Home (miles)", 1, 29, 10)

job_role = st.sidebar.selectbox("Job Role", [

    "Sales Executive", "Research Scientist", "Laboratory Technician",

    "Manufacturing Director", "Healthcare Representative", "Manager",

    "Sales Representative", "Research Director", "Human Resources"

])

business_travel = st.sidebar.selectbox("Business Travel", [

    "Non-Travel", "Travel_Rarely", "Travel_Frequently"

])

marital_status = st.sidebar.selectbox("Marital Status", ["Single", "Married", "Divorced"])


input_df = build_input_row(

    age, monthly_income, overtime, job_satisfaction, environment_satisfaction,

    total_working_years, years_at_company, distance_from_home,

    job_role, business_travel, marital_status, df_encoded, feature_cols

)


input_scaled = scaler.transform(input_df)

prob = model.predict_proba(input_scaled)[0][1]
```

```
prob_mean, ci_lo, ci_hi = bootstrap_prob_ci(input_df, boot_models, feature_cols)

label, color = risk_label_color(prob)


st.title("Employee Attrition Risk Dashboard")

st.markdown("**ECON 3916 Final Project — Dat Do** | IBM HR Analytics | Logistic Regression | ⚠️ Predictive output only — not for automated employment decisions.")

st.markdown("---")


col1, col2, col3 = st.columns(3)

with col1:

    st.metric("Attrition Probability", f"{prob:.1%}")

with col2:

    st.metric("95% Uncertainty Range", f"[{ci_lo:.1%}, {ci_hi:.1%}]")

with col3:

    st.markdown("### Risk Level")

    st.markdown(f"<h2 style='color:{color}'>{label}</h2>", unsafe_allow_html=True)


st.caption("Thresholds: Low < 20% | Medium 20-40% | High > 40%. Uncertainty range reflects 50-sample bootstrap resampling of training data — not a guarantee about any individual.")

st.markdown("---")


col_left, col_right = st.columns(2)


with col_left:

    st.subheader("Predicted Risk Gauge")
```

```
    fig, ax = plt.subplots(figsize=(6, 3))

    bar_color = {'green': '#2ecc71', 'orange': '#f39c12', 'red': '#e74c3c'}[color]

    ci_color = {'green': '#27ae60', 'orange': '#d68910', 'red': '#c0392b'}[color]

    ax.barh(['Risk'], [prob], color=bar_color, height=0.4, label=f'Predicted: {prob:.1%}')

    ax.barh(['Risk'], [ci_hi - ci_lo], left=ci_lo, color=ci_color, height=0.15, alpha=0.5,
label=f'95% range [{ci_lo:.1%}, {ci_hi:.1%}]')

    ax.axvline(0.20, color='gold', linestyle=':', linewidth=1.5, label='Low/Medium (20%)')

    ax.axvline(0.40, color='gray', linestyle='--', linewidth=1.5, label='Medium/High (40%)')

    ax.set_xlim(0, 1)

    ax.set_xlabel('Probability of Attrition')

    ax.set_title(f'Predicted Risk: {prob:.1%} — {label}', fontweight='bold', fontsize=13)

    ax.legend(fontsize=8, loc='lower right')

    ax.grid(axis='x', alpha=0.3)

    plt.tight_layout()

    st.pyplot(fig)

    plt.close()

    st.caption("Predictive output only — not causal evidence.")


with col_right:

    st.subheader("Top Predictors — This Profile")

    coef_series = pd.Series(model.coef_[0], index=feature_cols)

    contributions = coef_series * input_df.iloc[0]

    top_idx = contributions.abs().sort_values(ascending=False).head(10).index

    top_contrib = contributions[top_idx].sort_values()

    fig2, ax2 = plt.subplots(figsize=(6, 5))
```

```
    colors2 = ['#e74c3c' if v > 0 else '#3498db' for v in top_contrib.values]

    top_contrib.plot.barh(ax=ax2, color=colors2, edgecolor='black')

    ax2.axvline(0, color='black', linewidth=1)

    ax2.set_title('Feature Contributions\n⚠️ Predictive association, not causal effect',
fontweight='bold', fontsize=11)

    ax2.set_xlabel('Coefficient × Feature Value')

    ax2.grid(alpha=0.25)

    plt.tight_layout()

    st.pyplot(fig2)

    plt.close()

    st.caption("Red = increases predicted risk. Blue = decreases it. Not causal effects.")


st.markdown("---")

st.subheader("How Monthly Income Affects Attrition Risk")

income_range = np.arange(1000, 20500, 500)

probs_income = []

for inc in income_range:

    row = input_df.copy()

    if 'MonthlyIncome' in row.columns:

        row['MonthlyIncome'] = inc

    probs_income.append(model.predict_proba(scaler.transform(row))[0][1])

fig3, ax3 = plt.subplots(figsize=(10, 4))

ax3.plot(income_range, probs_income, color='steelblue', linewidth=2)

ax3.axvline(monthly_income, color='salmon', linestyle='--', linewidth=2, label=f'Current:
${monthly_income:,}')

ax3.axhline(0.40, color='gray', linestyle='--', linewidth=1, label='High threshold (40%)')
```

```
ax3.axhline(0.20, color='gold', linestyle=':', linewidth=1, label='Medium threshold (20%)')

ax3.fill_between(income_range, probs_income, alpha=0.1, color='steelblue')

ax3.set_xlabel('Monthly Income ($)', fontsize=12)

ax3.set_ylabel('Predicted Attrition Probability', fontsize=12)

ax3.set_title('Predicted Risk vs. Monthly Income\n(other inputs fixed at sidebar values)',
fontweight='bold', fontsize=13)

ax3.legend(fontsize=10)

ax3.grid(alpha=0.3)

plt.tight_layout()

st.pyplot(fig3)

plt.close()

st.caption("Predictive association only. Does not prove raising income reduces attrition.")


st.markdown("---")

st.subheader("Overtime vs. No Overtime — Predicted Risk")

row_no = build_input_row(age, monthly_income, "No", job_satisfaction,
environment_satisfaction, total_working_years, years_at_company, distance_from_home,
job_role, business_travel, marital_status, df_encoded, feature_cols)

row_yes = build_input_row(age, monthly_income, "Yes", job_satisfaction,
environment_satisfaction, total_working_years, years_at_company, distance_from_home,
job_role, business_travel, marital_status, df_encoded, feature_cols)

p_no = model.predict_proba(scaler.transform(row_no))[0][1]

p_yes = model.predict_proba(scaler.transform(row_yes))[0][1]

fig4, ax4 = plt.subplots(figsize=(6, 4))

bars = ax4.bar(['No Overtime', 'Overtime'], [p_no, p_yes], color=['#3498db', '#e74c3c'],
edgecolor='black', width=0.4)

for bar, val in zip(bars, [p_no, p_yes]):
```

```
    ax4.text(bar.get_x() + bar.get_width()/2, val + 0.01, f'{val:.1%}', ha='center',
fontweight='bold', fontsize=12)

ax4.axhline(0.40, color='gray', linestyle='--', linewidth=1, label='High threshold (40%)')

ax4.axhline(0.20, color='gold', linestyle=':', linewidth=1, label='Medium threshold (20%)')

ax4.set_ylim(0, 1)

ax4.set_ylabel('Predicted Attrition Probability')

ax4.set_title('Overtime vs. No Overtime\n(other inputs fixed at sidebar values)',
fontweight='bold', fontsize=12)

ax4.legend(fontsize=9)

ax4.grid(axis='y', alpha=0.3)

plt.tight_layout()

st.pyplot(fig4)

plt.close()

st.caption("Predictive association only — not causal evidence that eliminating overtime
reduces attrition.")



st.markdown("---")

st.markdown("**Model:** Logistic Regression | **Data:** IBM HR Analytics (N=1,470) |
**Split:** 80/20 stratified, random_state=42 | **CV F1:** 0.5552 | **Uncertainty:**
50-sample bootstrap | ⚠️ For HR risk-screening only. Not for automated employment
decisions.")
```

Here is the matching requirements.txt:

```
streamlit>=1.35.0

pandas>=2.2.3

numpy>=2.0.0

scikit-learn>=1.5.0

matplotlib>=3.9.0
```

Deployment instructions:

Test locally first by running `streamlit run app.py` in your terminal. Verify all sidebar sliders update the prediction, the income sweep chart moves the reference line, and the overtime bar chart updates. Then push app.py and requirements.txt to your GitHub repo. Go to streamlit.io/cloud, sign in with GitHub, click New app, select your repo and branch, set app.py as the main file, and click Deploy. Verify the deployed URL works in an incognito browser before submitting on Canvas.

Common deployment issues to watch for: If you see a pillow build error, your requirements.txt versions are too old for the Python version Streamlit Cloud picked — use minimum version constraints with >= rather than exact pins. If you see a TypeError on base_row initialization, replace base_row[:] = 0 with the pd.DataFrame dictionary constructor shown above. If startup is slow, reduce bootstrap iterations — 50 is the right number for Streamlit Cloud's free tier.

3. Five-page report writing prompt:

Disclaimer: I used this framework as a structural guide only. I wrote all prose myself and filled in the exact numbers from my notebook output. I verified that every metric in Table 1 and Table 2 matched my notebook's printed output before including them in the report. I rejected the framework's suggestion to include the correlation heatmap as a required figure, choosing instead to reference it in the text and prioritize the four figures most directly tied to the narrative.

INPUT:

[Prep] Act as an expert economics and machine learning writing advisor. I am writing the 5-page SCR-style report for my ECON 3916 final project on employee attrition prediction. I have already completed the notebook analysis and model results. I do not want you to write the full paper for me. I want you to help me build a clear report frame/outline that I can fill in with my own writing and results.

[Request] Create a structured 5-page report framework using the required sections: Problem Statement, Data Description, Methodology, Results, and Recommendation. For each section, tell me what to include, what order to present it in, and what evidence from my notebook should go there.

Use my actual project context:

- Dataset: IBM HR Analytics Employee Attrition dataset from Kaggle
- N = 1,470 employees
- Target: Attrition Yes/No
- Class imbalance: 16.1% left, 83.9% stayed
- Models compared: Logistic Regression and Random Forest
- Final model: Logistic Regression
- Main concern: prediction, not causation
- Main recommendation: use the model as an HR risk-screening tool, not an automatic decision system

Please provide:

1. A page-by-page structure for a 5-page report.
2. Section headings and subheadings.
3. Bullet points for what I should write under each section.
4. Suggested tables or figures to include.
5. Notes on where to discuss uncertainty, limitations, and feature-importance caveats.
6. Warnings about wording that might sound too causal or too confident.

Do not write the full report. Give me a framework only.

[Iterate] After you give the framework, I may ask you to expand one section, check my draft, shorten it to fit 5 pages, or make the wording more professional.

[Mechanism Check] Make sure the framework matches the assignment requirements exactly. The report must include: prediction question, stakeholder, decision it enables, data source/size/key features/quality assessment, models compared, train/test split, cross-validation, hyperparameter choices, performance metrics with confidence intervals, feature importance with caveats, actionable recommendation, uncertainty bounds, limitations, and next steps.

[Evaluate] Prioritize clarity, logic, and rubric alignment. The framework should help me write the paper myself while making sure I do not miss any required component or overclaim causality.

OUTPUT:

> Here is your 5-page SCR report framework. I am giving you the structure only — what to include, in what order, and what evidence from your notebook belongs in each section. Do not write beyond what your results actually show, and flag every feature importance claim with a causal caveat.
>
> ---
>
> **Page 1 — Problem Statement and Data Description (shared)**

Section 1: Problem Statement

Open with your one-sentence prediction question. State it exactly as you have it in the notebook — "Can we predict whether an employee will voluntarily leave the organization based on their demographic, compensation, and job-satisfaction characteristics?" Do not rewrite it into something that sounds causal.

Then write three short paragraphs in this order. First, identify the stakeholder and the specific decision the model enables — not just "an HR director" but what that director decides, which segments to target, with what interventions, prioritized by predicted risk and intervention cost. Second, explain what the model output is — an individual-level risk score used to prioritize outreach, not a final yes/no verdict. Third, write the causation disclaimer explicitly. Name a specific confounder — overtime and attrition may both be driven by job role or workload rather than overtime causing attrition. This disclaimer must appear in Section 1 and must be referenced again in Section 4 when you discuss feature importance.

Warning: Do not write "the model shows that overtime causes attrition" or "increasing income would reduce attrition." Any sentence that implies an intervention effect is a causal claim.

Section 2: Data Description

Start with the source — IBM HR Analytics Employee Attrition dataset from Kaggle, full URL, access date April 2026, N=1,470, 35 columns, 34 predictors plus binary target.

Then cover four subsections in this order.

Quality assessment: no missing values, MCAR/MAR/MNAR not applicable, four constant columns dropped. Keep this short — two to three sentences.

Class imbalance: 16.1% left (n=237), 83.9% stayed (n=1,233). Explain why this makes accuracy a misleading metric and why recall and F1 for the Left class are your primary criteria. Include Figure 1 — your attrition count and proportion chart — here.

Key EDA findings: report the overtime gap (30.5% vs 10.4%), the income median gap ($3,202 vs $5,204), and the correlation heatmap finding that all top correlates are negative and modest (max |r|=0.17). Include Figure 2 — your overtime stacked bar chart — here. One sentence noting that modest correlations mean the model must combine multiple weak signals.

Outlier assessment: IQR method from Chapter 4, four features with notable outliers — MonthlyIncome 7.8%, YearsAtCompany 7.1%, TotalWorkingYears 4.3%, NumCompaniesWorked 3.5% — retained because they reflect genuine workforce heterogeneity. Include Figure 3 — your four boxplots — here.

Page 2 — Methodology

Section 3: Methodology

Cover five subsections in this order.

Preprocessing: drop four columns, encode target as binary, one-hot encode nine categorical features with drop_first=True producing 44 predictors, StandardScaler fit on training set only with the fitted scaler applied to test set to prevent leakage.

Train/test split: 80/20, random_state=42, stratify=y. Report exact sample sizes — 1,176 training (16.2% attrition), 294 test (16.0% attrition). One sentence explaining why stratification matters given the 16% minority class.

Models compared: two models. Logistic Regression — simple, interpretable, parametric baseline. Random Forest with 200 trees — nonlinear ensemble capable of capturing feature interactions. One sentence explaining no heavy hyperparameter tuning was performed because the goal was a transparent comparison, not optimization.

Cross-validation: 5-fold stratified CV, StratifiedKFold with shuffle=True and random_state=42. For Logistic Regression, a pipeline wrapped StandardScaler inside each fold to prevent leakage — the scaler was refit on each fold's training portion separately. F1-score was the primary CV metric given class imbalance. Reference Chapter 15 here.

Confidence intervals: bootstrap 95% CIs using 1,000 resamples of the held-out test set for accuracy, precision, recall, F1, and AUC. Separate bootstrap function with single-class guard for AUC. One sentence explaining why CIs matter — a single test-set estimate has sampling variability and the CIs show whether the difference between models is reliable.

Pages 3 and 4 — Results

Section 4: Results

Open with one sentence orienting the reader — Logistic Regression outperforms Random Forest across all five metrics on the held-out test set and in every cross-validation fold.

Then present Table 1 — test set performance with 95% bootstrap CIs. Five rows: Accuracy, Precision, Recall, F1, AUC. Four columns: LR Mean, LR 95% CI, RF Mean, RF 95% CI. Use your exact notebook numbers. Do not round aggressively — keep four decimal places.

Then present Table 2 — 5-fold CV F1 results. Three columns: Model, Mean CV F1, Std. Two rows: LR (0.5552 ± 0.0884), RF (0.3044 ± 0.0834). Add per-fold scores if space allows.

Then write two paragraphs of interpretation. First paragraph: the recall gap is the most important finding — LR identifies 16 of 47 actual leavers (34.0%) while RF identifies only 4 (8.5%). Explain why recall matters more than accuracy in this HR context — false negatives are costly because they represent employees the firm fails to flag before they resign. Reference the confusion matrices. Note that non-overlapping bootstrap CIs for recall and F1 confirm the difference is not due to test-set sampling variation. Second paragraph: AUC CIs overlap slightly, so the strongest evidence for LR comes from recall, F1, and CV F1, not ranking performance.

Then write the feature importance subsection. Title it explicitly "Feature Importance — Predictive, Not Causal." Report the top three positive LR standardized coefficients — OverTime_Yes (0.865), BusinessTravel_Travel_Frequently (0.751), JobRole_Laboratory Technician (0.715) — and the top three negative — TotalWorkingYears (-0.556), EnvironmentSatisfaction (-0.482), JobSatisfaction (-0.419). Include Figure 4 — your LR coefficient plot. Write one sentence repeating the causation disclaimer — these coefficients reflect predictive weights, not policy levers.

Warning: Do not write "OverTime causes attrition." Do not write "employees with low satisfaction are more likely to leave." Write instead "OverTime_Yes is the strongest positive predictor of attrition risk in this model" and stop there.

Page 5 — Recommendation

Section 5: Recommendation

Cover six subsections in this order.

Actionable recommendation: pilot Logistic Regression as an HR risk-screening tool. Score active employees monthly. Use scores to prioritize voluntary,

non-punitive outreach — manager check-ins, workload reviews, career development conversations. The model supports human judgment, it does not replace it.

Priority segments: name the specific groups from your coefficient analysis — overtime workers, frequent travelers, Laboratory Technicians and Sales Representatives, single employees, employees with long gaps since last promotion. Tie each group to a specific positive coefficient. Do not claim these groups will definitely leave — say they share characteristics associated with higher predicted attrition risk in this dataset.

Uncertainty bounds: report the F1 bootstrap CI [0.29, 0.57] and the recall of 34.0%. Write one paragraph framing individual-level uncertainty — a predicted probability of 40% is a prioritization score, not a forecast. Employees with higher scores share characteristics associated with higher observed attrition in this training sample. The model is appropriate for broad screening, not for labeling individuals.

Threshold recommendation: the default 0.50 threshold is conservative and produces low recall. Suggest evaluating thresholds in the 0.25 to 0.30 range on a held-out validation set. Explain the tradeoff — lower thresholds increase recall at the cost of more false positives. For low-cost non-punitive interventions, this tradeoff is likely acceptable.

Limitations: at minimum cover three. First, generalizability — trained on one IBM dataset, may not transfer to other industries or firm sizes. Second, omitted variables — manager quality, external job offers, team culture, commute conditions, and recent life changes are not observed and may confound the results. Third, temporal stability — the model should be retrained periodically as workforce composition changes.

Next steps: three concrete steps. First, evaluate lower thresholds on a validation set using the firm's actual intervention cost structure. Second, collect firm-specific labeled data and retrain. Third, conduct exit interviews with model-flagged employees to validate whether they share the predicted risk factors, providing a partial check on predictive validity.

Warning: Do not end the report with a claim like "this model will reduce attrition by X%." You have no causal evidence for that. End with the model as a tool for prioritization under uncertainty, not a policy solution.

Figures to include across the report:

- Figure 1: Attrition count and proportion chart — Section 2, Class Imbalance
- Figure 2: Overtime stacked bar chart — Section 2, Key EDA Findings
- Figure 3: Four boxplots for outlier assessment — Section 2, Outlier Assessment
- Figure 4: LR standardized coefficient plot — Section 4, Feature Importance

Tables to include:

- Table 1: Test set performance with 95% bootstrap CIs — Section 4
- Table 2: 5-fold CV F1 scores — Section 4

Wording warnings throughout:

- Never use "causes," "leads to," "results in," or "if we change X then Y" unless you have a causal design
- Always write "is associated with higher predicted risk" or "is a strong positive predictor in this model"
- Always attribute feature importance to the model, not to the real world — "in this model, OverTime_Yes has the largest coefficient" not "overtime is the main driver of attrition"
- When discussing uncertainty, frame it as a range of plausible values, not as imprecision to apologize for — the CI is information, not a weakness